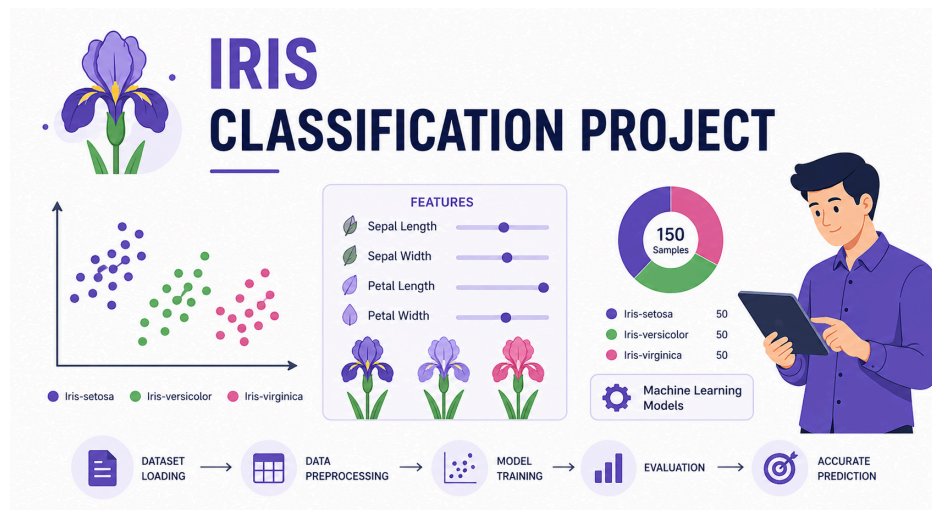


IRIS CLASSIFICATION PROJECT



Introduction

The Iris Classification project is a basic Machine Learning project used to classify iris flowers into different species. In this project, we use Python libraries such as NumPy, Pandas, and Scikit-learn to build models. The dataset is loaded from a CSV file named *Iris.csv*. The main aim is to train multiple classification algorithms and compare their performance. The features used for prediction are Sepal Length, Sepal Width, Petal Length, and Petal Width. The target variable is the species of the flower. The project follows a proper Machine Learning pipeline including data loading, preprocessing, training, and testing. Different models are trained using the same dataset. Accuracy is used to evaluate model performance. This project helps in understanding how classification algorithms work in real-world scenarios.

Dataset Description

The dataset used in this project is loaded using the Pandas library. It contains flower measurements and their corresponding species. The features selected in the code are SepalLengthCm, PetalLengthCm, SepalWidthCm, and PetalWidthCm. These are stored in variable **X** as input features. The target variable **y** contains the species names. The dataset is displayed using `head()` to check initial records. The shape of X and y is printed to understand the dataset size. The dataset is clean and structured, making it easy to process. It is commonly used for classification problems. Each row represents one flower sample. This dataset is ideal for testing multiple Machine Learning models.

Data Preprocessing

Data preprocessing begins by separating input features (X) and target variable (y). After that, the dataset is split into training and testing sets. The `train_test_split()` function is used for this purpose. In this project, 80% of the data is used for training and 20% for testing. The `random_state=0` ensures reproducibility of results. The lengths of training and testing datasets are printed. This helps verify the split. Proper splitting ensures the model is tested on unseen data. It improves model reliability. Preprocessing ensures the data is ready for training. This step is essential before applying Machine Learning algorithms.

Feature Scaling

Feature scaling is applied using the StandardScaler class from Scikit-learn. It standardizes the dataset so that all features have similar ranges. This is important because features like sepal length and petal width have different scales. Without scaling, larger values can dominate smaller ones. The scaler is first fitted on training data using `fit_transform()`. This calculates the mean and standard deviation. Then, the same transformation is applied to test data using `transform()`. This avoids data leakage. After scaling, the mean becomes 0 and standard deviation becomes 1. Feature scaling improves model performance. It is especially important for algorithms like SVM and Logistic Regression.

Formula:

$$Z = (X - \text{Mean}) / \text{Standard Deviation}$$

Machine Learning Models Used

1. Support Vector Machine (SVM)

Support Vector Machine is implemented using the `SVC` class from Scikit-learn. It is used to find the optimal boundary that separates different classes. In this code, the model is trained using `fit()` on scaled training data. After training, predictions are made using `predict()` on test data. SVM focuses on maximizing the margin between classes. It uses important data points called support vectors. It works well for both linear and non-linear classification. By default, it uses the RBF kernel. It performs very well on structured datasets like Iris. It is sensitive to feature scaling, which is why scaling is applied. The accuracy score shows its strong performance in this project.

2. Logistic Regression

Logistic Regression is implemented using the `LogisticRegression` class. It is used for classification by predicting probabilities. The model is trained using the training dataset. It uses a sigmoid function to map predictions between 0 and 1. Based on probability, it assigns a class label. After training, predictions are made on test data. It works best when the data is linearly separable. It is simple and fast to compute. Feature scaling improves its performance significantly. It provides interpretable results through coefficients. In this project, it achieves high accuracy due to simple dataset structure.

3. Naive Bayes

Naive Bayes is implemented using the `GaussianNB` class. It is based on probability and Bayes' theorem. It assumes that all features are independent of each other. The model is trained using training data and predicts using test data. It is very fast and efficient compared to other models. It works well with continuous data like the Iris dataset. It calculates the probability of each class and selects the highest one. Despite its assumptions, it performs well in many cases. It requires less data for training. It is less affected by irrelevant features. In this project, it provides good accuracy with fast execution.

4. Decision Tree Classifier

Decision Tree is implemented using the `DecisionTreeClassifier` class. It works by splitting data into branches based on feature values. Each split is based on a condition that maximizes information gain. The model is trained using training data and tested on unseen data. It creates a tree-like structure for decision-making. It is easy to understand and visualize. It does not require feature scaling. However, it can overfit the data if not controlled. It works well with both numerical and categorical data. In this project, it provides good classification results. It is useful for understanding how decisions are made step-by-step.

5. Random Forest Classifier

Random Forest is implemented using the `RandomForestClassifier` class. It is an ensemble method that combines multiple Decision Trees. Each tree is trained on a random subset of the dataset. The final prediction is made using majority voting. This improves accuracy and reduces overfitting. It is more robust compared to a single Decision Tree. The model is trained and tested similar to other algorithms. It can handle complex patterns in data. It also provides feature importance. It is widely used in real-world applications. In this project, it gives one of the highest accuracy scores.

Model Evaluation

Model evaluation is done using the accuracy score from Scikit-learn. The `accuracy_score()` function compares actual and predicted values. It calculates the percentage of correct predictions. Each model is evaluated on the test dataset. The results are printed in percentage format. This helps compare different models easily. Accuracy is a simple and effective metric for this dataset. Higher accuracy indicates better model performance. Each algorithm gives slightly different results. This helps in selecting the best model. Evaluation is an important step in Machine Learning.

Formula:

Accuracy = (Correct Predictions / Total Predictions)

Conclusion

In this project, multiple Machine Learning algorithms were implemented and tested. The dataset was successfully preprocessed and scaled. Models were trained and evaluated using accuracy scores. All algorithms performed well due to the simplicity of the dataset. Random Forest and SVM gave the best performance. Feature scaling improved the results significantly. The project helped in understanding the full Machine Learning workflow. It also showed how different models behave on the same dataset. This project is very useful for beginners. It builds a strong foundation in classification problems.

What I Learned

I learned how to load datasets using Pandas. I understood how to separate features and target variables. I learned how to split data into training and testing sets. I gained knowledge about feature scaling using StandardScaler. I implemented multiple Machine Learning algorithms using Scikit-learn. I learned how to train models using `fit()` and make predictions using `predict()`. I understood how to evaluate models using accuracy score. I improved my coding and debugging skills. I learned the importance of preprocessing in Machine Learning. Overall, this project improved my practical understanding of classification.